Course Code: CSA1741 **Course Name: ARTIFICIAL INTELLIGENCE**

Slot: B

ASSIGNMENT 2 – CO2

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: S. Arshad Ali **Reg. No:192412204**

CO2 Coverage: Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimization problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

Problem Context – Autonomous Campus Shuttle

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

- Predict how the three algorithms will behave under different map conditions
- Explain the algorithmic reason for your prediction
- Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Nodes expanded and nodes generated
- Maximum frontier size
- Path cost and whether the path is optimal
- Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

Design Decision

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

Evidence Expected: Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

Problem Context – Exploration Rover in an Unmapped Facility

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

Your Challenge

Design experiments to investigate the online agent under:

- Targets at different distances from the start
- Unreachable targets
- Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

Changed Condition

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

Evidence Expected: Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

Problem Context – Game Engine for a Board Game

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

Your Challenge

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs)
2. One game tree deliberately engineered so that almost no cutoffs occur
3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

- Predict the expected number of nodes visited
- Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs.

Inquiry

Compare prediction with experimental result. Explain:

- Why one tree requires substantially more work than another
- Which characteristics of move ordering create or prevent cutoffs
- What this reveals about evaluating an algorithm using only the branching factor and depth

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

Problem Context – Turn-Based Strategy Game

A game AI must choose its next move by looking ahead in a game with an average of b legal moves per turn.

Your Challenge

Design a depth-limited minimax solution with an evaluation function of your own.

Before implementation, determine:

- How many positions are examined for lookahead depth d
- How the evaluation function is used when the search is cut off
- How the number of terminal evaluations changes with d

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

- Number of nodes expanded and evaluation-function calls
- Execution time, where useful
- Quality of the move chosen (for example, win rate over 20 games against a random player)

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

Evidence Expected: Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

Problem Context – University Exam Timetabling

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

Design Starting Point

You are given only the following principle:

A value may be removed from a variable's domain if some neighboring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

Your Challenge

Convert this principle into a computational procedure. Determine:

- How the constraints are represented
- How a value without support is detected
- Which other pairs must be re-examined after a value is removed
- What condition indicates that the problem has no solution
- How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A timetable instance in which propagation solves or detects failure without any search
- A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

Correctness Inquiry

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

Problem Context – Delivery Tour Planning

A courier must visit n customers exactly once and return to the depot, minimising the total travel distance.

Your Challenge

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

Correctness Check: Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

Growth Investigation

Before experimentation, derive/predict how the number of candidate tours grows with n. Use at least three problem sizes to validate your prediction experimentally.

Local Search Alternative

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

Inquiry

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

Deployment Decision

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest n for which you would deploy exhaustive search, and the point at which you would switch to local search.

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

B. Define the Design Objective – 2 Marks

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

D. Predict Before Implementing – 1 Mark

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

E. Implement and Validate the Redesign – 2 Marks

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

F. Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

MARK DISTRIBUTION – ASSIGNMENT 2

Task	Assessed Area	Marks
Task 1	Uniform-cost, Greedy and A* search	6
Task 2	Online search agent	4
Task 3	Alpha-beta pruning	6
Task 4	Minimax and evaluation functions	6
Task 5	Constraint propagation and CSP	7
Task 6	Exhaustive and local search, optimisation	8
Task 7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Tasks 1–7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

Assessment Verification

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

ANSWERS

1.1 Aim

To compare Greedy Best-First Search (GBFS), Uniform Cost Search (UCS), and A* Search in different map environments and identify the most suitable search strategy for navigation.

1.2 Introduction

Search algorithms help intelligent agents find paths from an initial state to a goal state. Their performance depends on the environment, path costs, obstacles, and the heuristic used.

The three algorithms are:

Uniform Cost Search (UCS)

UCS expands the node with the lowest accumulated path cost.

$$f(n)=g(n)$$

Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal according to a heuristic.

$$f(n)=h(n)$$

A* Search

A* considers both the actual cost already traveled and the estimated cost remaining.

$$f(n)=g(n)+h(n)$$

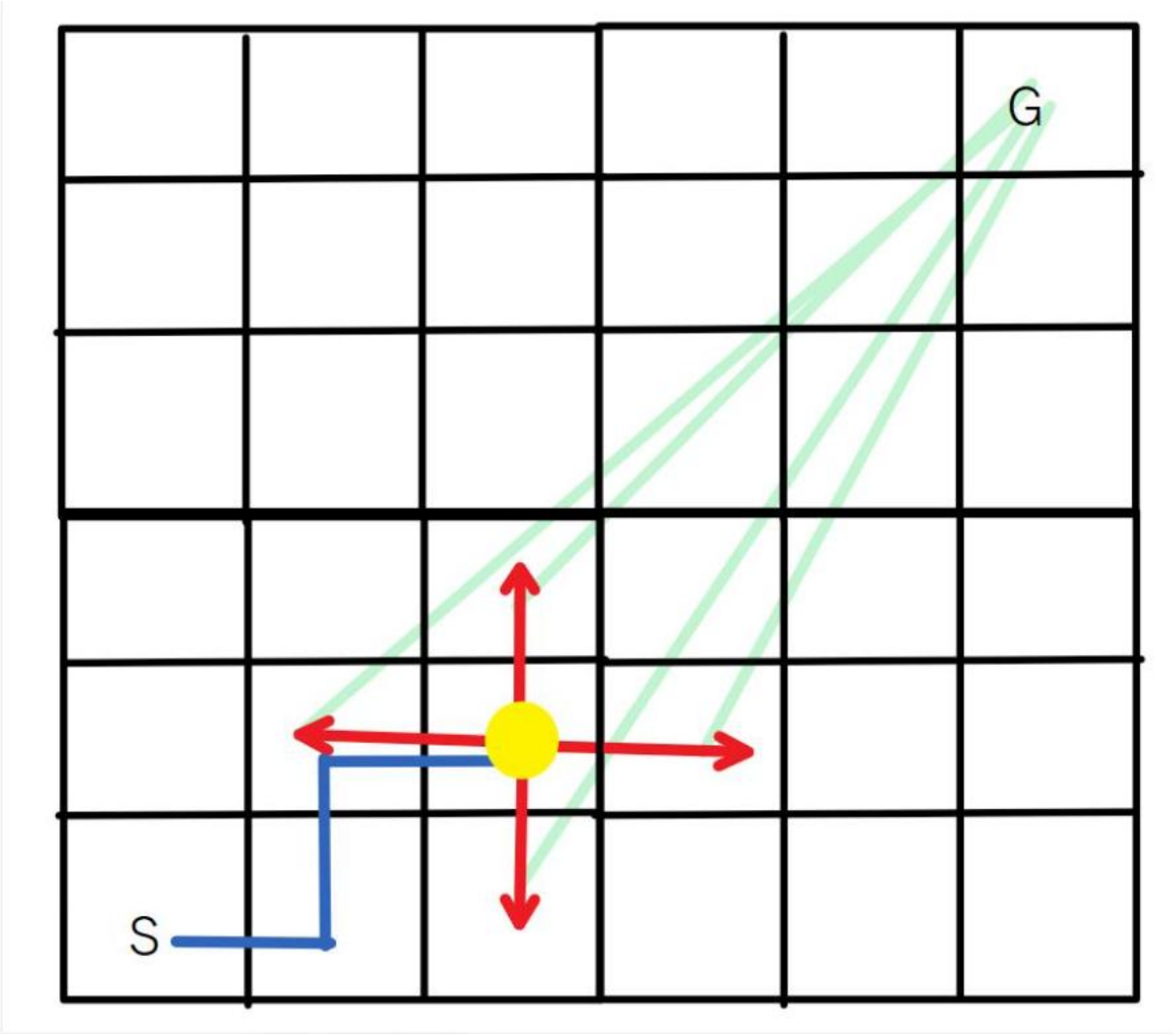
Where:

- $g(n)$ = cost from the start node to the current node

- \(h(n)\) = estimated cost from the current node to the goal

1.3 Map Environments

Map 1: Open Environment

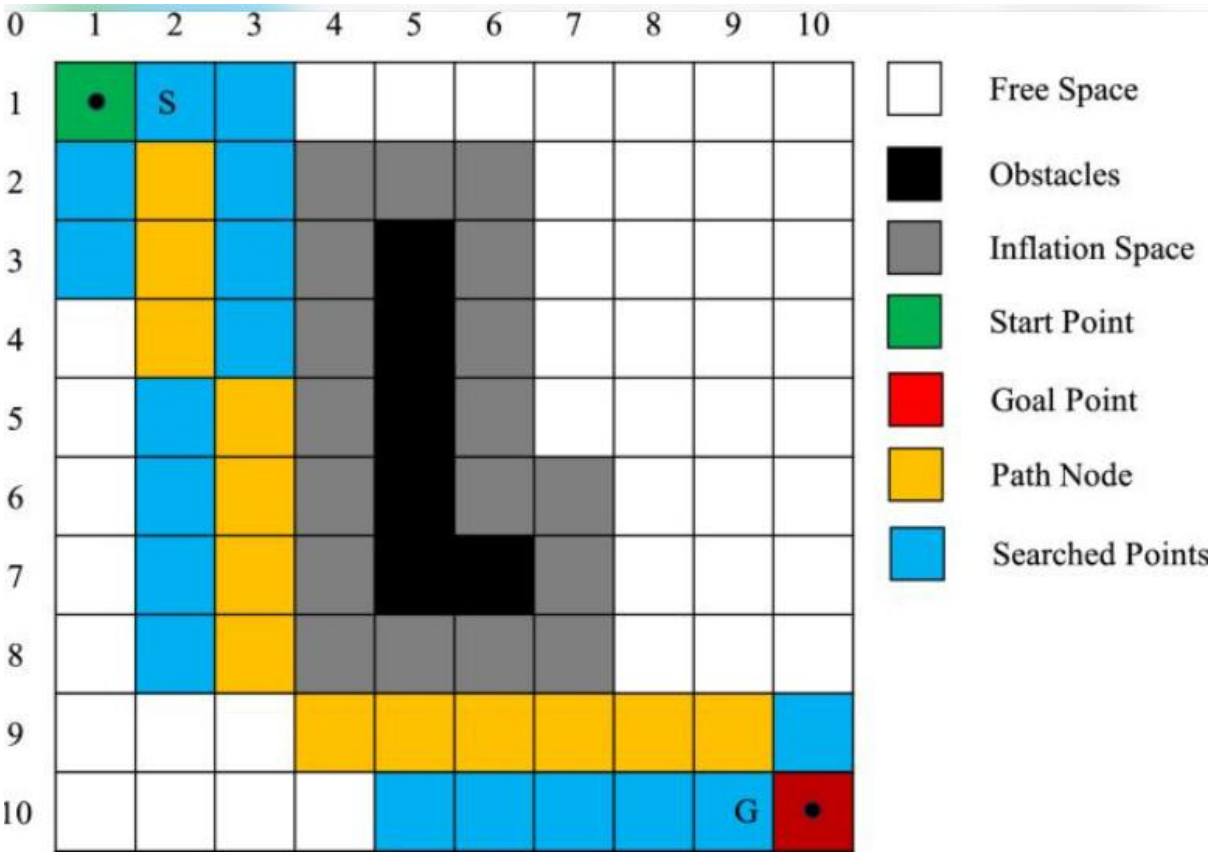


An open environment contains fewer obstacles and allows relatively direct movement toward the goal.

Expected behavior:

- GBFS reaches the goal quickly because the heuristic directs it toward the goal.
- UCS explores nodes according to path cost.
- A* balances path cost and estimated distance.

Map 2: Cluttered Environment



A dead-end environment contains paths that appear promising but do not lead to the goal.

Expected behavior:

- GBFS may enter a dead-end corridor because it looks closer to the goal.
- UCS systematically explores based on accumulated cost.
- A* can avoid unnecessary exploration when its heuristic provides useful guidance.

1.4 Heuristic Design

For a four-direction grid, Manhattan distance is suitable:

\[h(n)=|x_n-x_g|+|y_n-y_g| \]

This heuristic is admissible because it never overestimates the actual shortest path cost. It calculates the distance assuming there are no obstacles. Obstacles can only maintain or increase the actual path length.

1.5 Comparison Table

Algorithm	Evaluation	Advantage	Limitation
UCS	\(g(n)\)	Optimal for nonnegative costs	May expand many nodes
GBFS	\(h(n)\)	Often reaches goal quickly	Not always optimal
A*	\(g(n)+h(n)\)	Efficient and optimal with admissible heuristic	Requires memory for explored nodes

1.6 Experimental Evaluation

The following table represents the expected qualitative results. Actual values should be obtained by implementing and running the algorithms.

Environment	GBFS	UCS	A*
Open map	Fast goal discovery	More systematic	Efficient
Cluttered map	May take detours	Finds optimal route	Usually explores fewer nodes
Dead-end map	May enter dead ends	Reliable but expensive	Better guidance with heuristic

Measurements to record:

- Nodes expanded
- Nodes generated
- Maximum frontier size
- Total path cost
- Execution time
- Whether the solution is optimal

1.7 Recommendation

For a navigation system operating on 95% open roads with occasional detours, A* is recommended.

It combines actual traveled distance with estimated remaining distance. This helps the agent select efficient routes while still considering unexpected obstacles.

TASK 2: Online Search Agent in an Unknown Environment

2.1 Aim

To design and evaluate an online search agent that navigates an unmapped environment using limited information.

2.2 Introduction

An online search agent does not know the complete environment before starting. It must explore the surroundings, observe the results of its actions, and update its knowledge.

Examples include:

- Autonomous robots
- Rescue robots
- Exploration systems
- Robots operating in unfamiliar buildings

2.3 Online Depth-First Search

Online DFS explores one direction until it reaches a dead end. It then backtracks and explores another direction.

Algorithm

1. Start at the initial location.
2. Observe the available neighboring cells.
3. Select an unvisited neighboring cell.
4. Move to that cell.
5. Mark the cell as visited.
6. If no unvisited neighbor exists, backtrack.
7. Continue until the target is reached or all reachable cells are explored.

Pseudocode

```
ONLINE-DFS(location):
    mark location as visited

    if location is goal:
        return SUCCESS
```

```
for each available action:
    next = result(location, action)

    if next is not visited:
        execute action

        if ONLINE-DFS(next) == SUCCESS:
            return SUCCESS

        backtrack to location

return FAILURE
```

2.4 LRTA* (Learning Real-Time A*)

LRTA* improves online navigation by learning better heuristic estimates during exploration.

The agent:

1. Observes its current state.
2. Evaluates available neighboring states.
3. Chooses the action with the lowest estimated cost.
4. Updates its heuristic.
5. Moves and repeats the process.

A simplified update rule is:

\[H(s)=\min_{a}\left[c(s,a)+H(Result(s,a))\right] \]

Where:

- \(H(s)\) = updated heuristic value
- \(c(s,a)\) = cost of action

- \(Result(s,a)\) = next state after action \(a\)

2.5 Experimental Design

Test the agent under different conditions:

Experiment	Environment
1	Small maze with short distance
2	Large maze with long distance
3	Maze containing dead ends
4	Unreachable target
5	Repeated missions in the same environment

Record:

- Number of moves
- Number of revisited cells
- Number of explored cells
- Time taken
- Whether the target is reachable
- Competitive ratio, when the optimal offline path is known

The competitive ratio is:

\[CR=\frac{\text{Online agent's path cost}} {\text{Optimal offline path cost}} \]

2.6 Expected Comparison

Environment	Online DFS	LRTA*
Small maze	Simple exploration	Learns useful routes
Dead ends	May backtrack frequently	Can improve decisions
Repeated missions	Repeats exploration	Benefits from learned estimates
Unreachable goal	Explores reachable area	Also requires termination detection

2.7 Change in Scenario: 20 Rovers and 20 Static Facilities

When 20 rovers operate in 20 static facilities, each rover can learn the environment over repeated missions.

Recommended strategy: LRTA* with shared or stored knowledge, where appropriate.

Advantages:

- Reduced repeated exploration
- Improved route selection
- Faster performance in later missions
- Less unnecessary movement

However, coordination is needed to avoid collisions and congestion when multiple rovers operate simultaneously.

TASK 3: Minimax and Alpha-Beta Pruning

3.1 Aim

To implement Minimax and Alpha-Beta Pruning and analyze how node ordering affects pruning efficiency.

3.2 Introduction

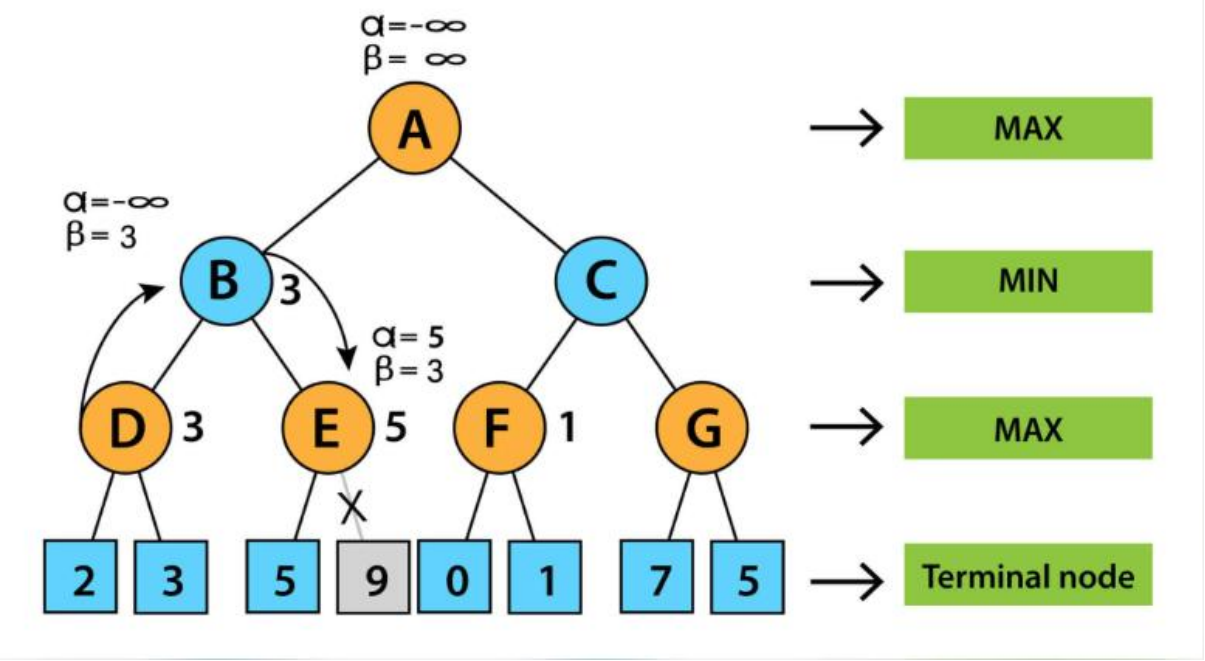
Minimax is an adversarial search algorithm used in two-player games.

- MAX attempts to maximize the score.
- MIN attempts to minimize the score.
- Both players are assumed to play optimally.

The minimax value is defined as:

\[Minimax(s)= \begin{cases} Utility(s), & \text{if }s\text{ is terminal}\\ \max Minimax(child), & \text{if MAX's turn}\\ \min Minimax(child), & \text{if MIN's turn} \end{cases} \]

3.3 Tree 1: Favorable Node Ordering



3.8 Node Count Analysis

For a full binary tree of depth 3:

\[N=1+2+4+8=15 \]

The number of leaves is:

\[L=2^3=8 \]

Minimax examines all 15 nodes in the complete tree.

Alpha-beta pruning:

- Best ordering: can approach \(O(b^{d/2})\)
- Worst ordering: \(O(b^d)\)

Where \(b\) is branching factor and \(d\) is search depth.

3.9 Comparison

Condition	Minimax	Alpha-Beta
Favorable ordering	Full tree	Strong pruning
Poor ordering	Full tree	Limited pruning
Equal leaf values	Full tree	Pruning depends on bounds

Decision quality	Optimal at searched depth	Same as minimax

3.10 Conclusion

Alpha-beta pruning produces the same optimal decision as Minimax but reduces the number of evaluated nodes. Its efficiency depends significantly on the ordering of actions.

TASK 4: Depth-Limited Minimax Strategy

4.1 Aim

To design a depth-limited Minimax algorithm and investigate the effect of branching factor and search depth on performance.

4.2 Node Count Derivation

For a complete tree with branching factor b and depth d:

$$ N=1+b+b^2+\cdots+b^d $$

Using the geometric series:

$$ \boxed{N=\frac{b^{d+1}-1}{b-1}} $$

The number of leaf nodes is:

$$ \boxed{L=b^d} $$

For a binary tree:

$$ N=2^{d+1}-1 $$

4.3 Evaluation Function

When the search reaches the depth limit, a heuristic evaluation function estimates the quality of the state.

For a simple game, a possible evaluation function is:

$$ Eval(s)=3P+2C+5W $$

Where:

- P = positional advantage
- C = control or centralization advantage
- W = winning-condition advantage

The coefficients are example weights and should be adjusted according to the game.

4.4 Depth-Limited Minimax

```
DEPTH-LIMITED-MINIMAX(state, depth):

    if terminal(state):
        return utility(state)

    if depth == 0:
        return evaluation(state)

    if MAX player:
        best = -infinity

        for each action:
            value = DEPTH-LIMITED-MINIMAX(
                result(state, action), depth-1)

            best = max(best, value)

        return best

    else:
        best = +infinity

        for each action:
            value = DEPTH-LIMITED-MINIMAX(
                result(state, action), depth-1)
```

```
        best = min(best, value)


    return best
```

4.5 Experimental Table

The following node counts are theoretical for a full tree.

Branching factor	Depth	Total nodes
2	3	15
2	5	63
3	3	40
3	5	364
4	3	85
4	5	1,365

Measure:

- Number of nodes expanded
- Number of evaluation calls
- Execution time
- Decision quality
- Win rate over repeated games

4.6 Response-Time Limit

For a real-time game, the agent must select a move before the response-time limit.

Recommended approach:

- Use depth-limited Minimax.
- Apply alpha-beta pruning.
- Use iterative deepening.

- Order promising moves first.
- Keep the best move from the last completed depth.

Iterative deepening gradually increases the search depth:

\[d=1,2,3,\ldots \]

If time runs out, the agent can return the best decision found at the previous completed depth.

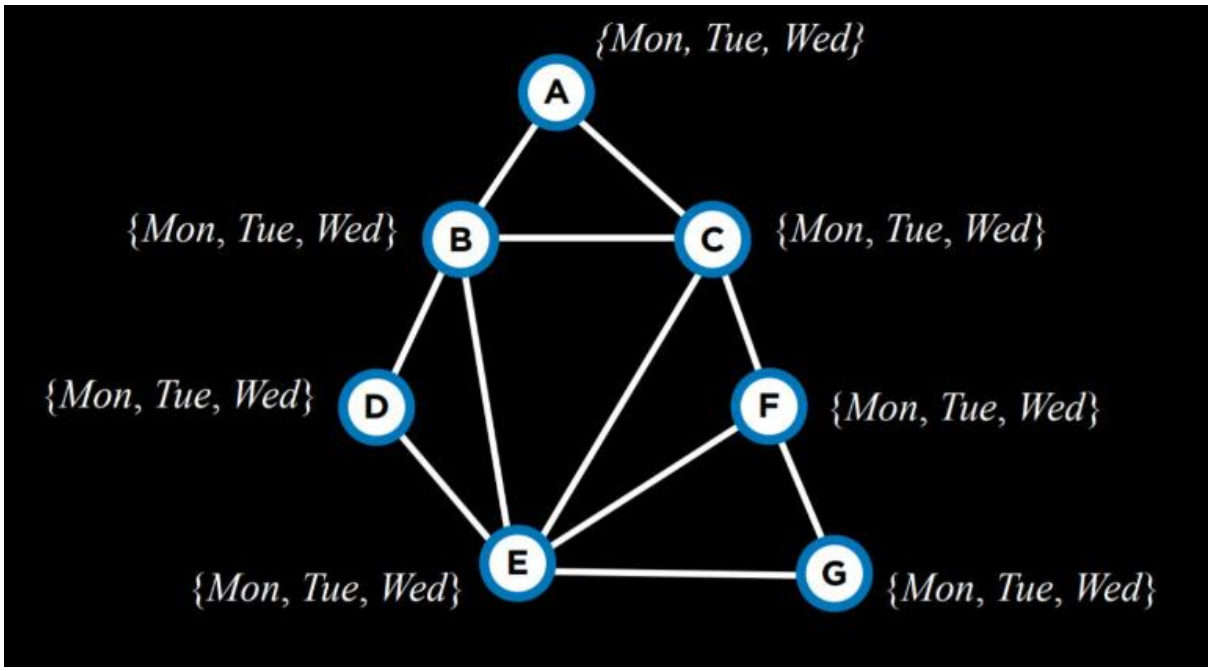
4.7 Scalability

The main limitation is exponential growth in the number of nodes.

For large branching factors:

- Deeper search requires more computation.
- Memory usage increases.
- Response time becomes unpredictable.

Alpha-beta pruning and improved evaluation functions help reduce the computational burden.



TASK 5: Constraint Satisfaction Problem and AC-3

5.1 Aim

To formulate an exam timetable as a Constraint Satisfaction Problem and derive constraint propagation using the AC-3 algorithm.

5.2 CSP Definition

A CSP consists of:

$$CSP=(X,D,C)$$

Where:

- X = variables
- D = domains of values
- C = constraints

Example: Exam Timetable

Variables:

$$X=\{E_1,E_2,E_3,E_4\}$$

Each variable represents an exam.

Domains:

$$D(E_i)=\{M,A,N\}$$

Where:

- M = Morning
- A = Afternoon
- N = Night

Constraints:

- Exams sharing students cannot occur simultaneously.
- Exams requiring the same hall cannot occur simultaneously.
- Some exams must occur before others.
- Certain exams cannot be assigned to specific time slots.

7.6 Expected Improvement

Metric	Original Minimax	Redesigned Search
Node expansion	Full tree	Reduced through pruning

Decision quality	Optimal at searched depth	Same at searched depth
Response time	May be high	Better controlled
Scalability	Limited	Improved
Memory usage	Depends on search	Still requires careful management

The actual improvement must be measured using the same test trees, branching factors, and depths.

7.7 Validation

The redesign should be tested using:

1. Favorable move ordering
2. Poor move ordering
3. Random move ordering
4. Different branching factors
5. Different search depths

Record the number of expanded nodes and execution time.

To verify correctness, compare the final decisions of Minimax and Alpha-Beta at the same depth. They should produce the same minimax value.

7.8 Defense of the Final Design

Alpha-beta pruning is selected because it eliminates branches that cannot influence the final decision while preserving Minimax's result.

Iterative deepening allows the agent to produce a usable decision even when a strict time limit is imposed.

Move ordering improves pruning efficiency, while the evaluation function allows the algorithm to make reasonable decisions without searching to terminal states.

Final conclusion: The redesigned strategy offers a practical balance between decision quality, computational efficiency, and response time.

OVERALL CONCLUSION

This assignment explored several important Artificial Intelligence search strategies and optimization techniques.

- UCS, Greedy Best-First Search, and A* were compared for pathfinding.
- Online DFS and LRTA* were studied for unknown environments.
- Minimax and Alpha-Beta Pruning were analyzed for adversarial decision-making.
- Depth-limited search was considered for controlling computational complexity.
- AC-3 constraint propagation was applied to timetable scheduling.
- Exhaustive search and local search were compared for the Traveling Salesperson Problem.
- A redesigned Minimax strategy was proposed using alpha-beta pruning and iterative deepening.

The overall finding is that no single search algorithm is suitable for every problem. The choice depends on the environment, optimality requirements, available memory, branching factor, and time constraints. Efficient AI systems combine appropriate search strategies, heuristics, and optimization methods to achieve reliable and practical performance.